

Computer Science Internal

Assessment: Criterion C

Total Word Count: 1,104

Techniques Used:

- OOP Principles
- Encapsulation
- Recommended Questions – Algorithmic Thinking
- Running Student Code – JDK-17

Object-Oriented Programming Principles:

Encapsulation:

Encapsulation was utilized in the development of this program to ensure data security of the students' and teachers' personal information regarding emails, passwords, and their academic profiles in their teacher's class and easier data management. Within this program, the object class variables and methods accessed only within the class are declared as private (See Figures C-1 and C-2) to prevent direct access from outside classes,

ensuring that sensitive user information is unable to be directly modified or accessed

```
public class User {  
  
    private int userID;  
    private String fullName;  
    private String email;  
    private String password;  
    private boolean isLoggedIn;  
    private String role;  
  
}
```

Figure C-1: Private Object Class Variables of User Class

```
// Add questions from alternative difficulty that haven't been completed  
for (Question question : allQuestions) {  
    if (question.getDifficulty().equals(AdditionalDifficulty) &&  
        !hasCompletedSuccessfully(question.getQuestionID()) &&  
        !recommendations.contains(question)) {  
        recommendations.add(question);  
    }  
}  
}  
}  
return recommendations;  
}  
/  
* Checks if a question has been successfully completed by the user  
* questionId = The ID of the question to check  
* returns true if the question has been completed successfully, false otherwise  
*/  
private boolean hasCompletedSuccessfully(int questionId) {  
    for (Attempt attempt : getAllAttempts()) {  
        if (attempt.getQuestionID() == questionId && attempt.getIsCorrect()) {  
            return true;  
        }  
    }  
    return false;  
}
```

Figure C-2: Example of Private Method Encapsulation within Student Object Class

```
protected int getStudentID() {
    return studentID;
}

protected void setStudentID(int studentID) {
    this.studentID = studentID;
}

protected int getClassID() {
    return classID;
}

protected void setClassID(int classID) {
    this.classID = classID;
}
```

Figure C-3: Student Object Class Protected Accessor and Mutator Methods

without proper validation, where the protected accessor and mutator methods (See Figure C-3) provides controlled access to private variables, allowing for data validation and maintaining data integrity. ArrayLists and methods accessed by the GUI within the object classes are declared as protected (See Figures C-4 and 5) to allow necessary access

within the same package while maintaining security from external packages, ensuring that data manipulation can only occur within the application's internal structure.

```
public class Student extends User {
    private int studentID;
    private int classID;
    private int gradeLevel;
    private String iLevel;
    private int correctAttempts;
    private int incorrectAttempts;
    protected ArrayList<Attempt> allAttempts = new ArrayList<>();
}
```

Figure C-4: Example of Protected ArrayList in Student Object Class

```
protected void loadStudentAttempts() throws SQLException {
    // Establish database connection
    Connection conn = DriverManager.getConnection(url);
    // Prepare SQL statement to get all attempts for this student
    PreparedStatement pstmt = conn.prepareStatement("SELECT * FROM Attempts WHERE studentID = ?");
    pstmt.setInt(1, this.getStudentID());
    ResultSet attemptData = pstmt.executeQuery();

    // Process each attempt record
    while (attemptData.next()) {
        // Parse the date string from database to LocalDateTime
        String dateString = attemptData.getString("dateCompleted").trim();
        LocalDateTime attemptDate = LocalDateTime.parse(dateString);
        // Create new Attempt object and add to student's attempt history
        Attempt att = new Attempt(attemptData.getInt("attemptID"),
                                   attemptData.getInt("studentID"),
                                   attemptData.getInt("questionID"),
                                   attemptData.getBoolean("is_Correct"),
                                   attemptDate);
        this.allAttempts.add(att);
    }
    conn.close();
}
```

Figure C-5: Example of GUI-Accessed method within the Student Object Class

Inheritance and Polymorphism:

Inheritance and Polymorphism were implemented in this program to promote code reusability and reduce redundancy. The User superclass contains common attributes and behaviors all system users share, [such as userID, fullName, email, and password](#) (See Figure C-6). [The Student and Teacher classes extend this User class](#), inheriting these basic user properties while adding their specific attributes and methods (See Figures C-7 and C-8). For example, the Student class includes additional attributes such as [studentID, gradeLevel, and an ArrayList for attempts](#), while the Teacher class incorporates [teacherID and an ArrayList for the teacher's classes](#). This inheritance structure eradicates the need to duplicate common code across the different User types (Student and Teacher), reducing redundancy and making the code more maintainable where changes only need to be applied once in the User parent class as they are automatically applied to all child classes. Polymorphism enhances this inheritance structure by [allowing Student and Teacher objects to be treated as User objects in the login system](#) (See Figures C-6, C-7, C-8 and C-9), stored in a single AllUsers collection in the Run class. The login method demonstrates this by returning a Student or Teacher object as a User reference. This combination of inheritance and polymorphism enables uniform handling of different user types while maintaining specific behaviors, creating a flexible and efficient system design.

```
public class User {
    private int userID;
    private String fullName;
    private String email;
    private String password;
    private boolean isLoggedIn;
    private String role;

    public User(int userID, String fullName, String email, String hashedPassword, String role) {
        this.userID = userID;
        this.fullName = fullName;
        this.email = email;
        this.password = hashedPassword;
        this.role = role;
        this.isLoggedIn = false;
    }
}

//Argon2 Code Source (See Source)
protected static User login(String email, String password) {
    // Search through all users
    for (User user : Run.AllUsers) {
        // Check if email matches (case insensitive)
        if (user.email.equalsIgnoreCase(email)) {
            // Create Argon2 hasher
            Argon2 argon2 = Argon2Factory.create();

            // Verify password matches stored hash
            if (argon2.verify(user.password, password.toCharArray())) {
                user.isLoggedIn = true;
                argon2.wipeArray(password.toCharArray()); // Clean up sensitive data
                return user; // Return user on successful login
            }
            argon2.wipeArray(password.toCharArray()); // Clean up sensitive data
            return null; // Return null if password incorrect
        }
    }
    return null; // Return null if email not found
}
```

Figure C-6: User Parent Class Constructor, Object Variables, and login method

```
public class Student extends User {
    private int studentID;
    private int classID;
    private int gradeLevel;
    private String iBLevel;
    private int correctAttempts;
    private int incorrectAttempts;
    protected ArrayList<Attempt> allAttempts = new ArrayList<>();

    private String url = "jdbc:sqlite:/Users/nicolemuzayyin/Library/Mobile Documents/com~apple~CloudDocs/eclipse_workspace_1

    public Student(int userID, String fullName, String email, String password, String role) {
        super(userID, fullName, email, password, role);
    }

    public Student(int userID, String fullName, String email, String password, String role, int studentID, int classID,
        int gradeLevel, String iBLevel, int correctAttempts, int incorrectAttempts) {
        super(userID, fullName, email, password, role);
        this.studentID = studentID;
        this.classID = classID;
        this.gradeLevel = gradeLevel;
        this.iBLevel = iBLevel;
        this.correctAttempts = correctAttempts;
        this.incorrectAttempts = incorrectAttempts;
    }

    public Student(int userID, String fullName, String email, String password, String role, int studentID,
        int gradeLevel, String iBLevel, int correctAttempts, int incorrectAttempts) {
        super(userID, fullName, email, password, role);
        this.studentID = studentID;
        this.classID = 0;
        this.gradeLevel = gradeLevel;
        this.iBLevel = iBLevel;
        this.correctAttempts = correctAttempts;
        this.incorrectAttempts = incorrectAttempts;
    }
}
```

Figure C-7: Student Child Class Constructors and Object Variables

```
public class Teacher extends User {
    private int teacherID;
    private String url = "jdbc:sqlite:/Users/nicolemuzayyin/Library/Mobile Documents/com~apple~CloudDocs/eclipse_

    protected ArrayList<Class> AllClasses = new ArrayList<>();

    public Teacher(int userID, String fullName, String email, String password, int teacherID, String role) {
        super(userID, fullName, email, password, role);
        this.teacherID = teacherID;
    }
}
```

Figure C-8: Teacher Child Class Constructors and Object Variables

```
// Login Button Action Listener method
private class LoginButtonListener implements ActionListener {
    @Override
    public void actionPerformed(ActionEvent e) {
        // Get user input from the text fields to retrieve username/email and password
        String email = usernameField.getText();
        String password = new String(passwordField.getPassword());

        // Attempt to login and get user object --> calling login method from User class
        User loggedInUser = User.login(email, password);

        // Check if login was successful
        if (loggedInUser != null) {
            // Show success message
            // JOptionPane Source (See Crit C Source 11)
            JOptionPane.showMessageDialog(Login_GUI.this, "Login successful!");

            // Route to appropriate menu based on user role
            if ("student".equals(loggedInUser.getRole())) {
                // Create and show student menu
                Student_MainMenu studentMenu = new Student_MainMenu(loggedInUser);
                studentMenu.setVisible(true);
                dispose(); // Close login window
            } else if ("teacher".equals(loggedInUser.getRole())) {
                // Create and show teacher menu
                Teacher_MainMenu teacherMenu = new Teacher_MainMenu(loggedInUser);
                teacherMenu.setVisible(true);
                dispose(); // Close login window
            }
        } else {
            // Show error message if login failed
            JOptionPane.showMessageDialog(Login_GUI.this,
                "Login failed. Please check your email and password.",
                "Login Error", JOptionPane.ERROR_MESSAGE);
        }
    }
}
```

Figure C-9: Implementation of polymorphic user handling in the login system, where different user types (Student/Teacher) are processed through a common User interface

Recommended Questions – Algorithmic Thinking:

This project required that students receive personalized question recommendations (Success Criterion 2.a.iii) based on their performance history when clicking the “Recommended Questions” button in the Student_QuestionBank GUI (Example seen in Figure C-10). The implementation utilizes JSwing's event-driven architecture to integrate the recommendation system with the GUI. The JSwing library was chosen for this program as it facilitates extensibility due to its vast documentation worldwide. The "Recommended Questions" button triggers the showRecommendations() method (See Figure C-11) through JSwing's ActionListener framework, leveraging the library's built-in event handling to maintain a clean separation between GUI interactions and the back-end. The DefaultTableModel (See Figure C-12) was selected for all GUI tables as it efficiently manages data and optimizes resource usage, enabling

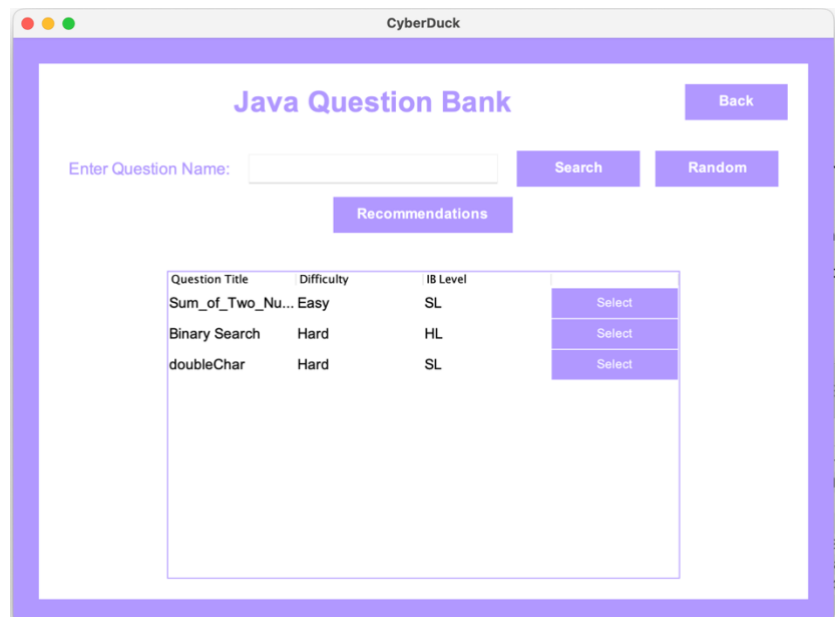


Figure C-10: GUI example showing the Java version of the Student_QuestionBank GUI

```
private void showRecommendations() {
    //Run getRecommendedQuestions method for the questions of the chosen coding language
    ArrayList<Question> recommendedQuestions = currentStudent.getRecommendedQuestions(relevantQuestions);
    table.setRowCount(0);
    if (recommendedQuestions.isEmpty()) {
        table.addRow(new Object[]{"No recommendations available at this time.", "-", "-", ""});
    } else {
        //update DefaultTableModel with the recommended questionx
        for (Question question : recommendedQuestions) {
            table.addRow(new Object[]{
                question.getQuestionTitle(),
                question.getDifficulty(),
                question.getIBLevel(),
                "Select"
            });
        }
    }
}
```

Figure C-11: showRecommendations() method in Student_QuestionBank GUI

```
// Define column structure for the question table
String[] columnNames = {"Question Title", "Difficulty", "IB Level", ""};

// Create custom table model with specific editing permissions
table = new DefaultTableModel(columnNames, 0) { // 0 indicates initial row count
    private static final long serialVersionUID = 1L;

    @Override
    public boolean isCellEditable(int row, int column) {
        // Only allow editing of the last column (index 3) which contains action buttons
        return column == 3;
    }
};

// Initialize JTable with custom model
questionTable = new JTable(table);
```

Figure C-12: DefaultTableModel use example

updates of personalized question recommendations without requiring complete GUI refreshes.

The implementation of JSwing's inheritance structure through `extends JButton` and implements `TableCellRenderer` in the `ButtonRenderer` and extends `DefaultCellEditor` and use of the composition with `Jbutton` in the `ButtonEditor` as it provided pre-built component behaviors, standardized rendering methods, and efficient event handling that enabled feasible table cell customization, reducing code complexity. The `ButtonRenderer` and `ButtonEditor` utilize JSwing's component model to manage button states, appearance, and

user interactions within table cells, eliminating the need for extensive custom code that would otherwise be required for basic button operations, cell rendering, and maintaining consistent GUI behavior across the application.

```

/**
 * Custom ButtonRenderer class that defines how the select buttons appear in
 * table cells Extends JButton to inherit button properties and implements
 * TableCellRenderer to customize how the cell is displayed in the table
 */
class ButtonRenderer extends JButton implements TableCellRenderer {
    private static final long serialVersionUID = 1L;

    // visual properties of buttons
    public ButtonRenderer() {
        setOpaque(true);
        setForeground(Color.WHITE);
        setBackground(LightPurple);
        setFocusPainted(false);
        setBorderPainted(true);
        setBorder(BorderFactory.createLineBorder(LightPurple, 1));
        setFont(new Font("Arial", Font.PLAIN, 12));
    }

    /**
     * Required method from TableCellRenderer interface Called for each cell in the
     * button column to determine how it should be displayed
     *
     * table = The JTable containing the button
     * value = The value to be rendered (button text)
     * isSelected = Whether the cell is selected
     * hasFocus = Whether the cell has focus
     * row = The row index of the cell
     * column = The column index of the cell
     * return the configured button component
     */
    public Component getTableCellRendererComponent(JTable table, Object value, boolean isSelected, boolean hasFocus,
        int row, int column) {
        setText((value == null) ? "" : value.toString());
        return this;
    }
}

/**
 * Custom ButtonEditor class -> handles button interactions in table cells
 * Extends DefaultCellEditor to provide editing capabilities for the button
 * cells
 */
class ButtonEditor extends DefaultCellEditor {
    private static final long serialVersionUID = 1L;
    protected JButton button;
    private String label;
    private boolean isPushed;

    // Constructor that initializes the button editor
    // checkBox = allows user to select specific rows
    public ButtonEditor(JCheckBox checkBox) {
        super(checkBox);
        button = new JButton();
        button.setOpaque(true);
        // Add action listener to handle button clicks
        button.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                fireEditingStopped(); // Signals that editing is complete
            }
        });
    }

    // retrieves the Question in the selected row
    public Object getCellEditorValue() {
        if (isPushed) {
            int selectedRow = questionTable.getSelectedRow();
            if (selectedRow != -1) {
                Question selectedQuestion = relevantQuestions.get(selectedRow);
                answerGUI(selectedQuestion);
            }
            isPushed = false;
            return label;
        }
        //if nothing is clicked, nothing is edited
        public boolean stopCellEditing() {
            isPushed = false;
            return super.stopCellEditing();
        }
    }
}

```

Figure C-13: ButtonRenderer and ButtonEditor code example

With this, an algorithm was needed to analyze student attempts and generate appropriate question recommendations based on difficulty levels and success rates (See Figure C-14). When `getRecommendedQuestions` is called, the program initializes tracking maps for attempts and failures at each difficulty level (Easy, Medium, Hard). This allows for comprehensive performance monitoring across all question difficulties.

A failure rate calculation system was implemented, maintaining separate counters for attempts and failures at each difficulty level. This algorithm first calculates failure rates for each difficulty level, determines the recommended difficulty based on performance thresholds (0.6 for progression to easier questions, 0.3 for harder questions), and generates a list of questions matching the recommended difficulty that has not been completed.

```
protected ArrayList<Question> getRecommendedQuestions(ArrayList<Question> allQuestions) {
    // For new users with no attempts, return all easy questions
    if (allAttempts.isEmpty()) {
        ArrayList<Question> easyQuestions = new ArrayList<>();
        for (Question q : allQuestions) {
            if (q.getDifficulty().equals("Easy")) {
                easyQuestions.add(q);
            }
        }
        return easyQuestions;
    }

    // Initialize tracking maps for attempts and failures at each difficulty level
    Map<String, Integer> attemptsPerDifficulty = new HashMap<>();
    Map<String, Integer> failsPerDifficulty = new HashMap<>();

    // Set initial counts to zero for all difficulty levels
    attemptsPerDifficulty.put("Easy", 0);
    attemptsPerDifficulty.put("Medium", 0);
    attemptsPerDifficulty.put("Hard", 0);
    failsPerDifficulty.put("Easy", 0);
    failsPerDifficulty.put("Medium", 0);
    failsPerDifficulty.put("Hard", 0);

    // Create a map for quick question lookup by ID
    Map<Integer, Question> questionMap = new HashMap<>();
    for (Question q : allQuestions) {
        questionMap.put(q.getQuestionID(), q);
    }

    // Calculate attempt and failure counts for each difficulty level
    for (Attempt attempt : allAttempts) {
        Question question = questionMap.get(attempt.getQuestionID());
        if (question != null) {
            String difficulty = question.getDifficulty();
            attemptsPerDifficulty.put(difficulty, attemptsPerDifficulty.get(difficulty) + 1);
            if (!attempt.getIsCorrect()) {
                failsPerDifficulty.put(difficulty, failsPerDifficulty.get(difficulty) + 1);
            }
        }
    }

    // Calculate failure rates for each difficulty level
    // If no attempts made at a difficulty level, failure rate is set to 0
    double easyFailRate = attemptsPerDifficulty.get("Easy") == 0 ? 0 :
        (double) failsPerDifficulty.get("Easy") / attemptsPerDifficulty.get("Easy");
    double mediumFailRate = attemptsPerDifficulty.get("Medium") == 0 ? 0 :
        (double) failsPerDifficulty.get("Medium") / attemptsPerDifficulty.get("Medium");
    double hardFailRate = attemptsPerDifficulty.get("Hard") == 0 ? 0 :
        (double) failsPerDifficulty.get("Hard") / attemptsPerDifficulty.get("Hard");

    // Determine recommended difficulty based on performance thresholds
    // Rules: Step down if failure rate > 60% with >= 3 attempts, Step up if failure rate < 30% with >= 5 attempts
    String recommendedDifficulty;
    if (hardFailRate > 0.6 && attemptsPerDifficulty.get("Hard") >= 3) {
        recommendedDifficulty = "Medium"; // Step down from Hard if struggling
    } else if (mediumFailRate > 0.6 && attemptsPerDifficulty.get("Medium") >= 3) {
        recommendedDifficulty = "Easy"; // Step down from Medium if struggling
    } else if (easyFailRate < 0.3 && attemptsPerDifficulty.get("Easy") >= 5) {
        recommendedDifficulty = "Medium"; // Step up from Easy if performing well
    } else if (mediumFailRate < 0.3 && attemptsPerDifficulty.get("Medium") >= 5) {
        recommendedDifficulty = "Hard"; // Step up from Medium if performing well
    } else {
        recommendedDifficulty = "Easy"; // Default to Easy if no conditions are met
    }

    // Filter questions matching the recommended difficulty that haven't been completed
    ArrayList<Question> recommendations = new ArrayList<>();
    for (Question question : allQuestions) {
        if (question.getDifficulty().equals(recommendedDifficulty) &&
            !hasCompletedSuccessfully(question.getQuestionID())) {
            recommendations.add(question);
        }
    }

    // If no questions available at recommended difficulty, find questions from an alternative difficulty level
    if (recommendations.isEmpty()) {
        String additionalDifficulty;
        if (recommendedDifficulty.equals("Hard")) {
            additionalDifficulty = "Medium"; // Step down from Hard
        } else if (recommendedDifficulty.equals("Medium")) {
            additionalDifficulty = "Easy"; // Step down from Medium
        } else {
            additionalDifficulty = "Medium"; // Step up from Easy
        }

        // Add questions from alternative difficulty that haven't been completed
        for (Question question : allQuestions) {
            if (question.getDifficulty().equals(additionalDifficulty) &&
                !hasCompletedSuccessfully(question.getQuestionID()) &&
                !recommendations.contains(question)) {
                recommendations.add(question);
            }
        }
    }

    return recommendations;
}

/**
 * Checks if a question has been successfully completed by the user
 * @param questionID The ID of the question to check
 * @return true if the question has been completed successfully, false otherwise
 */
private boolean hasCompletedSuccessfully(int questionID) {
    for (Attempt attempt : getAllAttempts()) {
        if (attempt.getQuestionID() == questionID && attempt.getIsCorrect()) {
            return true;
        }
    }
    return false;
}
```

Figure C-14: Code of Recommended Questions Algorithm in Student Class

Running Student Code – JDK-17 and Jython:

The program implements a system for executing both Java and Python code submissions, requiring careful algorithmic consideration to ensure secure and efficient code execution (See Figures 16-18). Jython was implemented into the runPythonCode() method (See Figure C-17) due to its vast online documentation and extensible architecture, enabling easy implementation of additional features and custom modules, ensuring reliable code execution, eliminating the need for additional Python runtime installations on the system and permitting extensibility.

The runJavaCode method (See Figure C-18) utilizes JDK-17 as a standalone installation to ensure consistent code execution and environment control, independent of IDE configurations while leveraging its enhanced

security features for reliable code assessment. To maintain system integrity, ProcessBuilder implements secure isolation of code execution, effectively preventing malicious code from accessing or damaging the program. The implementation employs Java.nio.file for reliable temporary file management, providing consistent file operations and enhanced error handling to prevent data corruption during simultaneous access.

CS_IA_Nicole_Muzayyin	Today at 1:11 AM	-- Folder
> bin	Today at 1:09 AM	-- Folder
> data	Today at 1:11 AM	-- Folder
> jdk-17-jdk	Today at 1:00 AM	-- Folder
> lib	Today at 12:31 AM	-- Folder
> src	Today at 1:08 AM	-- Folder

Figure C-15: Project files – showing Java Development Kit Location

```
/**
 * Main method to execute code based on language selection Determines whether to
 * run Python or Java code based on question language
 */
protected String runCode(String code, Map<String, Object> parameters, String fileName) {
    if (language.equalsIgnoreCase("Python")) {
        return runPythonCode(code, parameters);
    } else if (language.equalsIgnoreCase("Java")) {
        return runJavaCode(code, parameters, fileName);
    } else {
        return "Error: Unsupported language type: " + language;
    }
}
```

Figure C-16: Implementation of runCode method

```
/**
 * Executes Python code using Jython interpreter Key features: - Creates new
 * Python interpreter instance - Captures output using ByteArrayOutputStream -
 * Injects parameters into Python environment - Constructs and evaluates
 * function call - Handles Python-specific exceptions
 */
protected String runPythonCode(String code, Map<String, Object> parameters) {
    try {
        PythonInterpreter interpreter = new PythonInterpreter();
        // Set up output capture
        ByteArrayOutputStream outputStream = new ByteArrayOutputStream();
        interpreter.setOut(outputStream);

        // Execute the submitted code
        interpreter.exec(code);

        // Inject parameters into Python environment
        for (Map.Entry<String, Object> entry : parameters.entrySet()) {
            interpreter.set(entry.getKey(), entry.getValue());
        }

        // Construct function name from question title
        String functionName = questionTitle.replaceAll("\\s+", "_").toLowerCase();

        // Build function call string with parameters
        StringBuilder callStr = new StringBuilder(functionName + "(");
        boolean first = true;
        for (String paramName : parameters.keySet()) {
            if (!first) {
                callStr.append(", ");
            }
            callStr.append(paramName);
            first = false;
        }
        callStr.append(")");

        // Evaluate function and return result
        PyObject result = interpreter.eval(callStr.toString());

        return result != null ? result.toString() : outputStream.toString().trim();
    } catch (PyException e) {
        return "Python Error: " + e.value;
    } catch (Exception e) {
        return "Error: " + e.getMessage();
    }
}
```

Figure C-17: Implementation of runPythonCode method

Finally, [BufferedReader](#) and [IO](#) classes complete the security chain by optimizing memory usage through efficient stream processing, safeguarding against system crashes from large output handling.

```
protected String runJavaCode(String code, Map<String, Object> parameters, String fileName) {
    // Java File and Java Path Source (See Sources and)
    //Installing JDK-17 and Code Source(See Sources and)

    //Defines path to JDK by getting absolute path of created file
    String projectDir = new File("").getAbsolutePath();
    String jdkPath = projectDir + "/jdk-17.jdk/Contents/Home";
    String javacPath = jdkPath + "/bin/javac";
    String javaPath = jdkPath + "/bin/java";

    // Verify JDK installation
    File javacFile = new File(javacPath);
    File javaFile = new File(javaPath);
    if (!javacFile.exists() || !javaFile.exists()) {
        return "Error: JDK not found. Please ensure JDK is properly installed.";
    }

    try {
        // Get method name from question title, replacing spaces with underscores
        String methodName = this.getQuestionTitle().replaceAll("\\s+", "_");

        // Verify the submitted code contains the required method
        if (!code.contains(methodName)) {
            return "Error: Method '" + methodName + "' not found in submitted code";
        }

        // Create temporary directory for compilation and execution
        Path tempDir = Files.createTempDirectory("java_execution");

        // Extract class name from file name (removing .java extension)
        String className = fileName.substring(0, fileName.lastIndexOf('.'));

        // Prepare the complete source code by wrapping user code in a class
        StringBuilder preparedCode = new StringBuilder();
        preparedCode.append("public class ").append(className).append(" {\n");
        preparedCode.append(code).append("\n");

        // Add main method to call user's method with provided parameters
        preparedCode.append("    public static void main(String[] args) {\n");
        preparedCode.append("        ").append(className).append(" solution = new ").append(className)
            .append("();\n");
        preparedCode.append("        System.out.println(solution. ").append(methodName).append("(");

        // Add parameter parsing for each parameter in the method call
        int paramIndex = 0;
        for (Map.Entry<String, Object> entry : parameters.entrySet()) {
            if (paramIndex > 0) {
                preparedCode.append(", ");
            }

            // Handle different parameter types (Integer, Double, String)
            if (entry.getValue() instanceof Integer) {
                preparedCode.append("Integer.parseInt(args[ ").append(paramIndex).append("])");
            } else if (entry.getValue() instanceof Double) {
                preparedCode.append("Double.parseDouble(args[ ").append(paramIndex).append("])");
            } else {
                preparedCode.append("args[ ").append(paramIndex).append("]");
            }
            paramIndex++;
        }

        preparedCode.append("));\n");
        preparedCode.append("    }\n");
    }
}
```

```

// Write the complete source code to a file
Path javaFilePath = tempDir.resolve(fileName);
Files.write(javaFilePath, preparedCode.toString().getBytes());

// Compile the Java file
ProcessBuilder compileBuilder = new ProcessBuilder(javacPath, javaFilePath.toString());
compileBuilder.redirectErrorStream(true);
Process compileProcess = compileBuilder.start();

// Capture compilation output/errors
StringBuilder compileOutput = new StringBuilder();
try (BufferedReader reader = new BufferedReader(new InputStreamReader(compileProcess.getInputStream()))) {
    String line;
    while ((line = reader.readLine()) != null) {
        compileOutput.append(line).append("\n");
    }
}

// Check if compilation was successful
int compileResult = compileProcess.waitFor();
if (compileResult != 0) {
    return "Compilation Error: " + compileOutput.toString();
}

// Prepare command to run the compiled program
ArrayList<String> command = new ArrayList<>();
command.add(javaPath);
command.add("-cp");
command.add(tempDir.toString() + File.pathSeparator + jdkPath + "/lib/*");
command.add(className);

// Add parameters as command line arguments
for (Object value : parameters.values()) {
    command.add(value.toString());
}

// Set up and start the process to run the program
ProcessBuilder runBuilder = new ProcessBuilder(command);
runBuilder.redirectErrorStream(true);
runBuilder.directory(tempDir.toFile());
Process runProcess = runBuilder.start();

// Capture program output
StringBuilder output = new StringBuilder();
try (BufferedReader reader = new BufferedReader(new InputStreamReader(runProcess.getInputStream()))) {
    String line;
    while ((line = reader.readLine()) != null) {
        output.append(line).append("\n");
    }
}

// Wait for completion with timeout
boolean completed = runProcess.waitFor(10, TimeUnit.SECONDS);
if (!completed) {
    runProcess.destroyForcibly();
    return "Error: Execution timed out";
}

// Clean up temporary files
Files.walk(tempDir).sorted(Comparator.reverseOrder()).map(Path::toFile).forEach(File::delete);

// Return the program output
return output.toString().trim();
} catch (IOException e) {
    return "Error: IO Exception - " + e.getMessage();
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
    return "Error: Execution interrupted";
} catch (Exception e) {
    return "Error: " + e.getMessage();
}
}

```

Figure C-18: Implementation of runJavaCode method

Database Operations:

This program required database implementation where the program could add, modify, and delete data whilst maintaining data security. SQLite was implemented due to its cross-platform flexibility, allowing users on all platforms, including macOS and Windows, and its self-contained nature, requiring no server setup or configuration while ensuring reliable data operations. To further ensure data security, prepared statements and parameterized queries were consistently used across all database operations to prevent SQL injection attacks (See Examples in Figures 19-21). This includes proper input validation, sanitization of parameters, and the use of bound variables (marked by ? placeholders) instead of direct string concatenation when constructing SQL queries. Additionally, database connections and resources are properly managed with explicit closing of PreparedStatement and Connection objects to prevent resource leaks.

```
/**
 * Deletes a test case from the database
 * testcaseID = ID of the test case to delete
 */
protected void deleteTestCase(int testcaseID) throws SQLException {
    //Using prepared statement to prevent SQL injection
    Connection conn = DriverManager.getConnection(url);
    PreparedStatement pstmt = conn
        .prepareStatement("DELETE FROM 'Test Cases' WHERE testcaseID = ? AND questionID = ?");

    // Ensures deletion only occurs for valid testcase and question combination
    pstmt.setInt(1, testcaseID);
    pstmt.setInt(2, this.questionID);

    // Execute delete and handle the result
    int affectedRows = pstmt.executeUpdate();
    if (affectedRows > 0) {
        // Remove the test case from the in-memory collection
        question_testCases.removeIf(testCase -> testCase.getTestCaseID() == testcaseID);
        System.out.println("Test case deleted successfully.");
    } else {
        System.out.println("No test case found with ID: " + testcaseID + " for question ID: " + this.questionID);
    }

    // Properly close database resources
    pstmt.close();
    conn.close();
}
```

Figure C-19: Example of Delete Query
Interaction with CyberDuck Database –
Deleting TestCases


```

/**
 * Updates an existing test case with new parameter values and expected output
 * parameterValues = New parameter values for the test case
 * expectedOutput = New expected result
 * testCaseId = test case to alter
 */
protected void updateTestCase(int testCaseId, ArrayList<String> parameterValues, String expectedOutput) throws SQLException {

    // Using prepared statement to prevent SQL injection
    Connection conn = DriverManager.getConnection(url);
    PreparedStatement pstmt = conn.prepareStatement(
        "UPDATE 'Test Cases' SET questionParameters = ?, expectedValue = ?, expectedValueDataType = ? WHERE testCaseID = ?"

    // Convert parameter values to JSON format using the correct parameter names
    JSONObject parametersJson = new JSONObject();

    // Iterate through the question parameters and their corresponding values
    for (int i = 0; i < questionParameters.size() && i < parameterValues.size(); i++) {
        String paramName = questionParameters.get(i).getParameterVariable();
        String paramValue = parameterValues.get(i);
        String paramType = questionParameters.get(i).getDataType();

        if (paramValue != null && !paramValue.isEmpty()) {
            try {
                // Handle different data types
                switch (paramType.toLowerCase()) {
                    case "int":
                        parametersJson.addProperty(paramName, Integer.parseInt(paramValue));
                        break;
                    case "double":
                        parametersJson.addProperty(paramName, Double.parseDouble(paramValue));
                        break;
                    case "long":
                        parametersJson.addProperty(paramName, Long.parseLong(paramValue));
                        break;
                    case "boolean":
                        parametersJson.addProperty(paramName, Boolean.parseBoolean(paramValue));
                        break;
                    default:
                        // Remove quotes if they exist at start and end
                        if (paramValue.startsWith("\"") && paramValue.endsWith("\"")) {
                            paramValue = paramValue.substring(1, paramValue.length() - 1);
                        }
                        parametersJson.addProperty(paramName, paramValue);
                }
            } catch (NumberFormatException e) {
                // If parsing fails, store as string
                parametersJson.addProperty(paramName, paramValue);
            }
        }
    }

    // Determine data type of expected output
    String expectedValueDataType;
    try {
        Integer.parseInt(expectedOutput);
        expectedValueDataType = "int";
    } catch (NumberFormatException e) {
        try {
            Long.parseLong(expectedOutput);
            expectedValueDataType = "long";
        } catch (NumberFormatException e2) {
            try {
                Double.parseDouble(expectedOutput);
                expectedValueDataType = "double";
            } catch (NumberFormatException e3) {
                expectedValueDataType = "String";
            }
        }
    }

    // Set the parameters for the SQL update
    pstmt.setString(1, parametersJson.toString());
    pstmt.setString(2, expectedOutput);
    pstmt.setString(3, expectedValueDataType);
    pstmt.setInt(4, testCaseId);

    try {
        // Execute update and handle the result
        int affectedRows = pstmt.executeUpdate();

        if (affectedRows > 0) {
            // Update the in-memory test case
            for (Test_Case testCase : question_testCases) {
                if (testCase.getTestCaseID() == testCaseId) {
                    // Convert JSON to Map for Test_Case object
                    Gson gson = new Gson();
                    Type mapType = new TypeToken<Map<String, Object>>() {}.getType();
                    Map<String, Object> parameters = gson.fromJson(parametersJson.toString(), mapType);

                    testCase.setParameters(parameters);
                    testCase.setExpectedValue(expectedOutput);
                    testCase.setExpectedValueDataType(expectedValueDataType);
                    break;
                }
            }
            System.out.println("Test case updated successfully.");
        } else {
            System.out.println("No test case found with ID: " + testCaseId);
        }
    } finally {
        pstmt.close();
        conn.close();
    }
}

```

Figure C-20: Example of Update Query Interaction with CyberDuck Database – Updating TestCases


```

/**
 * Adds a new test case to the database with parameter values and expected
 * output parameterValues = List of parameter values for the test case
 * expectedOutput = Expected result of the test case
 */
protected void addTestCase(ArrayList<String> parameterValues, String expectedOutput) throws SQLException {
    // Using prepared statement to prevent SQL injection
    Connection conn = DriverManager.getConnection(url);
    PreparedStatement pstmt = conn.prepareStatement(
        "INSERT INTO 'Test Cases' (questionID, questionParameters, expectedValue, expectedValueDataType) VALUES (?, ?, ?, ?)",
        Statement.RETURN_GENERATED_KEYS);
    // Convert parameter values to JSON format
    // Validates parameters for null/empty values before adding
    JSONObject parametersJson = new JSONObject();
    for (int i = 0; i < parameterValues.size(); i++) {
        String paramValue = parameterValues.get(i);
        String paramVariable = questionParameters.get(i).getParameterVariable(); // Get the parameter variable

        if (paramValue != null && !paramValue.isEmpty()) {
            // Handle different data types appropriately
            try {
                // Try parsing as number first
                double numValue = Double.parseDouble(paramValue);
                if (paramValue.contains(".")) {
                    parametersJson.addProperty(paramVariable, numValue);
                } else {
                    parametersJson.addProperty(paramVariable, (int)numValue);
                }
            } catch (NumberFormatException e) {
                // If not a number, add as string
                // Remove quotes if they exist at start and end
                if (paramValue.startsWith("\"") && paramValue.endsWith("\"")) {
                    paramValue = paramValue.substring(1, paramValue.length() - 1);
                }
                parametersJson.addProperty(paramVariable, paramValue);
            }
        }
    }
    // Determine data type of expected output through type checking
    // Validates and sanitizes expected output type
    String expectedValueDataType;
    try {
        Integer.parseInt(expectedOutput);
        expectedValueDataType = "int";
    } catch (NumberFormatException e) {
        try {
            Long.parseLong(expectedOutput);
            expectedValueDataType = "long";
        } catch (NumberFormatException e2) {
            try {
                Double.parseDouble(expectedOutput);
                expectedValueDataType = "double";
            } catch (NumberFormatException e3) {
                expectedValueDataType = "String";
            }
        }
    }

    // Using parameterized queries to prevent SQL injection
    pstmt.setInt(1, this.questionID);
    pstmt.setString(2, parametersJson.toString());
    pstmt.setString(3, expectedOutput);
    pstmt.setString(4, expectedValueDataType);

    // Execute update and handle the result
    int affectedRows = pstmt.executeUpdate();
    if (affectedRows > 0) {
        // Retrieve and handle the generated test case ID
        ResultSet generatedKeys = pstmt.getGeneratedKeys();
        if (generatedKeys.next()) {
            int testCaseID = generatedKeys.getInt(1);

            // Convert JSON parameters to Map for Test_Case object
            Gson gson = new Gson();
            Type mapType = new TypeToken<Map<String, Object>>() {
            }.getType();
            Map<String, Object> parameters = gson.fromJson(parametersJson.toString(), mapType);

            // Create and add new test case to the collection
            Test_Case newTestCase = new Test_Case(testCaseID, this.questionID, parameters, expectedOutput,
                expectedValueDataType);
            question_testCases.add(newTestCase);
        }
    }

    // Properly close database resources to prevent resource leaks
    pstmt.close();
    conn.close();
}

```

Figure C-21: Example of Add Query Interaction with CyberDuck Database – Adding TestCases

Bibliography – (Code Sources):

1. adminoctopuscodes, 'Convert String to Other Data Types in Java',
OctopusCodes, 2023 <<https://octopuscodes.com/java/java-tutorial/basic-java/convert-string-to-other-data-types-in-java/>> [accessed 27 November 2024]
2. 'CheckBoxList (Oracle Fusion Middleware Java API Reference for Oracle Extension SDK Reference)'
<<https://docs.oracle.com/middleware/11119/jdev/api-reference-esdk/oracle/ide/controls/checkboxlist/CheckBoxList.html>> [accessed 28 November 2024]
3. 'Component Class in Java - GeeksforGeeks'
<<https://www.geeksforgeeks.org/component-class-in-java/>> [accessed 28 November 2024]
4. '———' <<https://www.geeksforgeeks.org/component-class-in-java/>> [accessed 28 November 2024]
5. 'DateTimeFormatter (Java Platform SE 8)'
<<https://docs.oracle.com/javase/8/docs/api/java/time/format/DateTimeFormatter.html>> [accessed 27 November 2024]
6. 'DefaultListCellRenderer (Java Platform SE 8)'
<<https://docs.oracle.com/javase/8/docs/api/javax/swing/DefaultListCellRenderer.html>> [accessed 28 November 2024]

7. 'FileNameExtensionFilter (Java Platform SE 8)'
<<https://docs.oracle.com/javase/8/docs/api/index.html?javax/swing/filechooser/FileNameExtensionFilter.html>> [accessed 27 November 2024]
8. H, Moritz, 'Phxql/Argon2-Jvm', 2024 <<https://github.com/phxql/argon2-jvm>> [accessed 27 November 2024]
9. 'Installation of the JDK on macOS'
<<https://docs.oracle.com/en/java/javase/17/install/installation-jdk-macos.html?locale=en#GUID-2FE451B0-9572-4E38-A1A5-568B77B146DE>> [accessed 27 November 2024]
10. 'Java JFrame', GeeksforGeeks, 2023 <<https://www.geeksforgeeks.org/java-jframe/>> [accessed 27 November 2024]
11. 'Java JOptionPane - GeeksforGeeks' <<https://www.geeksforgeeks.org/java-joptionpane/>> [accessed 27 November 2024]
12. 'Java JScrollPane - GeeksforGeeks' <<https://www.geeksforgeeks.org/java-jscrollpane/>> [accessed 27 November 2024]
13. 'Java String Reference' <https://www.w3schools.com/java/java_ref_string.asp> [accessed 27 November 2024]
14. 'Java Swing | JFileChooser', GeeksforGeeks, 2018
<<https://www.geeksforgeeks.org/java-swing-jfilechooser/>> [accessed 27 November 2024]

15. 'Java Swing | JProgressBar', *GeeksforGeeks*, 2018
<<https://www.geeksforgeeks.org/java-swing-jprogressbar/>> [accessed 27 November 2024]
16. 'Java Swing | JSpinner - GeeksforGeeks' <<https://www.geeksforgeeks.org/java-swing-jspinner/>> [accessed 28 November 2024]
17. 'Java Swing | JSplitPane with Examples', *GeeksforGeeks*, 2018
<<https://www.geeksforgeeks.org/java-swing-jsplitpane-examples/>> [accessed 27 November 2024]
18. 'Java.Time.LocalDateTime Class in Java', *GeeksforGeeks*, 2021
<<https://www.geeksforgeeks.org/java-time-localdatetime-class-in-java/>> [accessed 27 November 2024]
19. 'JDK 17 Documentation', *Oracle Help Center*
<<https://docs.oracle.com/en/java/javase/17/?locale=en>> [accessed 27 November 2024]
20. 'Map Interface in Java - GeeksforGeeks' <<https://www.geeksforgeeks.org/map-interface-java-examples/>> [accessed 27 November 2024]
21. 'Modern File Input/Output with Java Path API and Files Helper Methods'
<<https://blogs.oracle.com/javamagazine/post/path-files-input-output?locale=en>> [accessed 27 November 2024]

22. 'MouseListener and MouseMotionListener in Java - GeeksforGeeks'

<<https://www.geeksforgeeks.org/mouselistener-mousemotionlistener-java/>>

[accessed 28 November 2024]

23. Parakhin, Konstantin, 'Jython: Run Python Code inside Java', *Medium*, 2021

<<https://medium.com/@KosteRico/jython-run-python-code-inside-java-de4d12f227ae>> [accessed 27 November 2024]

24. 'SpinnerNumberModel (Java Platform SE 8)'

<<https://docs.oracle.com/javase/8/docs/api/index.html?javax/swing/SpinnerNumberModel.html>> [accessed 28 November 2024]

25. 'StringBuilder Class in Java with Examples - GeeksforGeeks'

<<https://www.geeksforgeeks.org/stringbuilder-class-in-java-with-examples/>>

[accessed 27 November 2024]

26. 'TableColumn (Java Platform SE 8)'

<<https://docs.oracle.com/javase/8/docs/api/index.html?javax/swing/table/TableColumn.html>> [accessed 27 November 2024]

27. 'Tips4java/Source/ButtonColumn.Java at Main · Tips4java/Tips4java', *GitHub*

<<https://github.com/tips4java/tips4java/blob/main/source/ButtonColumn.java>>

[accessed 27 November 2024]

28. 'Understanding DefaultTableModel in Java', *ThoughtCo*

<<https://www.thoughtco.com/defaulttablemodel-overview-2033890>> [accessed 27 November 2024]

29. 'Graphics2D (Java Platform SE 8)'

<<https://docs.oracle.com/javase/8/docs/api/java/awt/Graphics2D.html>>

[accessed 9 December 2024]